RT-Thread PWM 开发指南

文件标识：RK-KF-YF-C32

发布版本：V1.1.0

日期：2025-07-30

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

脉宽调制（PWM，Pulse Width Modulation）功能在嵌入式系统中是非常常见的，它是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，广泛应用在从测量、通信到功率控制与变换的许多领域中。本文主要介绍 Rockchip 平台 PWM 的基本特性和使用方法。

产品版本

芯片名称	RT-Thread 版本
RK2108	lts-v3.1.x
RK3308	lts-v3.1.x/lts-v4.1.x
RK1808	lts-v3.1.x
RV1109/RV1126	lts-v3.1.x
RK356X	lts-v3.1.x
RK3588	lts-v4.1.x
RV1103/RV1106	lts-v3.1.x
RK3562	lts-v4.1.x
RK3576	lts-v4.1.x
RK2118	lts-v4.1.x
RK3506	lts-v4.1.x
RV1103B	lts-v3.1.x
RK1820	lts-v5.1.x
RK2116	lts-v4.1.x

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	丁凌崧	2024.11.22	初始发布
V1.1.0	丁凌崧	2025.07.30	添加 RK1820/RK2116 相关说明和优化《功能支持》章节 HTML 表格

目录

RT-Thread PWM 开发指南
1. 功能支持
2. 驱动目录
3. 配置说明
4. 软件应用
    4.1 Framework
    4.2 Continous
    4.3 Oneshot
    4.4 Capture
    4.5 Global control
    4.6 Output offset
    4.7 Counter
    4.8 Frequency meter
    4.9 IR input

1. 功能支持

SOC	PWM Version	PWM Feature											
		Channel Number	Continuous	Oneshot	Capture	Global Control	Output Offset	Counter	Frequency Meter	IR Output	IR Input	Wave Generator	Biphasic Counter
RK2108	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3308	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3288	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK322X/ RK312XH	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK356X	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3588	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RV1103/ RV1106	v3	4	Y	Y	Y	N	Y	Y	N	Y	Y	N	N
RK3562	v3	4	Y	Y	Y	N	Y	Y	N	Y	Y	N	N
RK3576	v4(PWM0)	2	Y	Y	Y	Y	Y	N	N	N	Y	N	N
	v4(PWM1)	6	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y
	v4(PWM2)	8	Y	Y	Y	Y	Y	N	N	Y	Y	N	Y
RK2118	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N
RV1103B	v4(PWM0)	4	Y	Y	Y	Y	Y	Y	Y	N	Y	N	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	N	N	N	N	N	N
	v4(PWM2)	4	Y	Y	Y	Y	Y	N	N	N	N	N	N
RK3506	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y	N
	v4(PWM1)	8	Y	Y	Y	Y	Y	Y	Y	Y	N	N	Y
RK1820	v4(PWM0)	8	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	N
RK2116	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y (channel1~3)	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	Y	Y	N	N	N	N

- PWM v2
  - 每个控制器的 channel 数为 4；
  - continuous/oneshot/caputure：每个控制器的任一通道均可以支持；
  - IR input：仅有每个控制器的 channel 3 可以支持。
- PWM v3：
  - counter/output offset：每个控制器的任一通道均可以支持；
  - 其他功能与 PWM v2 相同。
- PWM v4：
  - 不同控制器的 channel 数可能不相同，单个控制器最大支持的 channel 数为 8；
  - continuous/oneshot/capture/global control/output offset：每个控制器的任一通道均可以支持；
  - counter/frequency meter/IR output/IR input：部分控制器可以支持，且单个控制器同时只有一个 channel 可以使能上述功能；
  - wave generator：部分控制器可以支持，且单个控制器中的多个 channel 可以同时使能该功能；
  - biphasic counter：部分控制器可以支持，且单个控制器中有多个 channel 可以支持该功能。

2. 驱动目录

RT-Thread PWM 设备框架：

```
components/drivers/include/drivers/rt_drv_pwm.h
components/drivers/misc/rt_drv_pwm.c
```

Rockchip 平台 PWM 设备驱动：

```
bsp/rockchip/common/drivers/drv_pwm.c
```

3. 配置说明

以 RK3506 平台为例，在 bsp/rockchip/rk3506-32 目录下通过 `scons --menuconfig` 进行 RT-Thread 配置。PWM 功能需使能的配置如下：

RT-Thread PWM 设备框架配置 RT_USING_PWM，对应的 Help 信息为：

```
Symbol: RT_USING_PWM [=n]
Type  : boolean
Prompt: Using PWM device drivers
  Location:
    -> RT-Thread Components
      -> Device Drivers
  Defined at ../../../components/drivers/Kconfig:131
```

Rockchip 平台 PWM 设备驱动配置 RT_USING_PWM0/RT_USING_PWM1，对应的 Help 信息为：

```
Symbol: RT_USING_PWM0 [=n]
Type  : boolean
Prompt: Enable PWM0
  Location:
    -> RT-Thread rockchip RK3506 drivers
      -> Enable PWM
  Defined at ./driver/Kconfig:96
  Depends on: RT_USING_PWM [=n]

Symbol: RT_USING_PWM1 [=n]
Type  : boolean
Prompt: Enable PWM1
  Location:
    -> RT-Thread rockchip RK3506 drivers
      -> Enable PWM
  Defined at ./driver/Kconfig:100
  Depends on: RT_USING_PWM [=n]
```

4. 软件应用

对于 RT-Thread PWM 的应用方法在官方文档 《PWM 设备》 已有说明，本章节主要简要介绍下 PWM 框架并针对 Rockchip 平台的 PWM 特性进一步展开说明。

4.1 Framework

- RT-Thread PWM 框架主要通过 `struct rt_pwm_configuration` 来配置 PWM 设备：

```
enum rt_pwm_aligned_mode
{
    PWM_LEFT_ALIGNED = 1,
    PWM_CENTER_ALIGNED,
    PWM_UNALIGNED,
};

struct rt_pwm_configuration
{
    rt_uint32_t channel; /* 1-n or 0-n, which depends on specific MCU requirements */
    rt_uint32_t period;  /* unit:ns 1ns~4.29s:1Ghz~0.23hz */
    rt_uint32_t pulse;   /* unit:ns (pulse<=period) */
    rt_uint32_t count;   /* (count+1) repeated effective periods */
    rt_uint32_t offset;  /* unit:ns 0ns<=offset<=(period-duty) */
    rt_uint32_t mask;    /* channel mask for lock/unlock, such as 0x5 indicates channel0 and
channel2 */

    rt_bool_t polarity;  /* polarity inverted or not */
    /*
     * RT_TRUE  : The channel of pwm is complememtary.
     * RT_FALSE : The channel of pwm is nomal.
    */
    rt_bool_t  complementary;

    rt_uint32_t delay;    /* unit:ms, which indicates timer val in freqency meter mode */
    rt_uint32_t freqency; /* unit:hz, which indicates freqency meter result */

    rt_uint64_t counter_res; /* The number of effective waveforms */

    enum rt_pwm_aligned_mode aligned; /* aligned mode */
};
```

  - 配置项 count/offset/mask/delay/frequency/aligned 为 Rockchip 平台新增。
- RT-Thread PWM 框架提供的应用接口：

```
rt_err_t rt_device_pwm_register(struct rt_device_pwm *device, const char *name, const struct
rt_pwm_ops *ops, const void *user_data);
rt_err_t rt_pwm_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_set(struct rt_device_pwm *device, int channel, rt_uint32_t period, rt_uint32_t
pulse);
rt_err_t rt_pwm_set_period(struct rt_device_pwm *device, int channel, rt_uint32_t period);
rt_err_t rt_pwm_set_pulse(struct rt_device_pwm *device, int channel, rt_uint32_t pulse);
// RK 平台新增
rt_err_t rt_pwm_set_internal(struct rt_device_pwm *device, int channel, rt_uint32_t period,
rt_uint32_t pulse, rt_uint32_t polarity, enum rt_pwm_aligned_mode aligned);
rt_err_t rt_pwm_set_offset(struct rt_device_pwm *device, int channel, rt_uint32_t offset);
rt_err_t rt_pwm_set_oneshot(struct rt_device_pwm *device, int channel, rt_uint32_t count);
rt_err_t rt_pwm_set_capture(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_lock(struct rt_device_pwm *device, rt_uint32_t channel_mask);
rt_err_t rt_pwm_unlock(struct rt_device_pwm *device, rt_uint8_t channel_mask);
rt_err_t rt_pwm_int_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_int_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_get_result(struct rt_device_pwm *device, int channel, rt_uint64_t *cnt_res);
rt_err_t rt_pwm_set_freqency_meter(struct rt_device_pwm *device, int channel, rt_uint32_t delay_ms,
rt_uint32_t *freq);
```

  - 上述各应用接口在 Rockchip 平台是通过 bsp/rockchip/common/drivers/drv_pwm.c 中的 rockchip_pwm_control() 函数结合各 PWM CMD 来实现：

```
struct rt_pwm_ops
{
    rt_err_t (*control)(struct rt_device_pwm *device, int cmd, void *arg);
};

#define PWM_CMD_ENABLE      (RT_DEVICE_CTRL_BASE(PWM) + 0)
#define PWM_CMD_DISABLE     (RT_DEVICE_CTRL_BASE(PWM) + 1)
#define PWM_CMD_SET         (RT_DEVICE_CTRL_BASE(PWM) + 2)
#define PWM_CMD_GET         (RT_DEVICE_CTRL_BASE(PWM) + 3)
#define PWMN_CMD_ENABLE     (RT_DEVICE_CTRL_BASE(PWM) + 4)
#define PWMN_CMD_DISABLE    (RT_DEVICE_CTRL_BASE(PWM) + 5)
#define PWM_CMD_SET_PERIOD  (RT_DEVICE_CTRL_BASE(PWM) + 6)
#define PWM_CMD_SET_PULSE   (RT_DEVICE_CTRL_BASE(PWM) + 7)
// RK 平台新增
#define PWM_CMD_SET_ONESHOT (RT_DEVICE_CTRL_BASE(PWM) + 8)
#define PWM_CMD_SET_OFFSET  (RT_DEVICE_CTRL_BASE(PWM) + 9)
#define PWM_CMD_SET_CAPTURE (RT_DEVICE_CTRL_BASE(PWM) + 10)
#define PWM_CMD_LOCK        (RT_DEVICE_CTRL_BASE(PWM) + 11)
#define PWM_CMD_UNLOCK      (RT_DEVICE_CTRL_BASE(PWM) + 12)
#define PWM_CMD_INT_ENABLE  (RT_DEVICE_CTRL_BASE(PWM) + 13)
#define PWM_CMD_INT_DISABLE (RT_DEVICE_CTRL_BASE(PWM) + 14)
#define PWM_CMD_CNT_ENABLE  (RT_DEVICE_CTRL_BASE(PWM) + 15)
#define PWM_CMD_CNT_DISABLE (RT_DEVICE_CTRL_BASE(PWM) + 16)
#define PWM_CMD_GET_CNT_RES (RT_DEVICE_CTRL_BASE(PWM) + 17)
#define PWM_CMD_SET_FREQ    (RT_DEVICE_CTRL_BASE(PWM) + 18)
```

- 使能 RT_USING_FINSH 配置后，可以在命令行通过 `pwm` 指令获取到所有测试指令的 Usage 信息：

```
msh />pwm
Usage:
pwm probe   <device name>                                    - probe pwm by name
pwm enable  <channel>                                        - enable pwm channel
pwm disable <channel>                                        - disable pwm channel
pwm get     <channel>                                        - get pwm channel info
pwm set     <channel> <period> <pulse> <polarity> <aligned>  - set pwm channel info
pwm offset  <channel> <offset>                               - set pwm channel offset
pwm oneshot <channel> <count>                                - set pwm oneshot mode
pwm capture <channel>                                        - set pwm capture mode
pwm lock    <channel_mask>                                   - enable pwm global lock
pwm unlock  <channel_mask>                                   - disable pwm global lock
pwm int_enable  <channel>                                    - enable pwm interrupt
pwm int_disable <channel>                                    - disable pwm interrupt
pwm cnt_enable  <channel>                                    - enable pwm counter
pwm cnt_disable <channel>                                    - disable pwm counter
pwm cnt_get_res <channel>                                    - get pwm counter result
pwm set_freq    <channel> <mdelay>                           - set pwm frequency meter
```

4.2 Continous

连续输出模式，支持持续输出指定占空比的 PWM 波形。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set(pwm_device, channel, period, pulse);
/* to config polarity and aligned */
// result = rt_pwm_set_internal(pwm_device, channel, period, pulse, polarity, aligned);
result = rt_pwm_enable(pwm_device, channel);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。
- period：周期，单位 ns。
- pulse：占空比，单位 ns。
- polarity：波形极性，0 表示 normal，1 表示 inverted。
- aligned：对齐模式。

命令行测试指令：

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm enable 0
```

4.3 Oneshot

单次输出模式，支持输出指定个数的 PWM 波形。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set(pwm_device, channel, period, pulse);
/* to config polarity and aligned */
// result = rt_pwm_set_internal(pwm_device, channel, period, pulse, polarity, aligned);
result = rt_pwm_set_oneshot(pwm_device, channel, count);
result = rt_pwm_enable(pwm_device, channel);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。
- period：周期，单位 ns。
- pulse：占空比，单位 ns。
- polarity：波形极性，0 表示 normal，1 表示 inverted。
- aligned：对齐模式。
- count：波形数量。

命令行测试指令：

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm oneshot 0 10
pwm enable 0
```

4.4 Capture

输入捕获模式，支持计算输入波形高低电平的持续时间。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_int_enable(pwm_device, channel);
result = rt_pwm_set_capture(pwm_device, channel);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。

命令行测试指令：

```
pwm probe pwm0
pwm int_enable 0
pwm capture 0
```

4.5 Global control

全局控制模式，支持多通道配置的同步更新，结合 continous/oneshot mode 可以实现输出同步、互补输出等功能。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_lock(pwm_device, channel_mask);
......
result = rt_pwm_unlock(pwm_device, channel_mask);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel_mask：通道 mask，如 0x3 表示 channel0 和 channel1。

命令行测试指令：

```
pwm probe pwm0
pwm lock 3
pwm set 0 10000 2000 0 0
pwm set 1 10000 2000 0 0
pwm oneshot 0 10
pwm oneshot 1 10
pwm unlock 3
```

4.6 Output offset

输出偏移模式，支持 PWM 输出波形偏移指定的时间，通常结合 global control 在 oneshot 模式下使用，以实现更复杂的波形输出功能。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
...
result = rt_pwm_set_offset(pwm_device, channel, offset);
...
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。
- offset：偏移时间，单位 ns。

命令行测试指令：

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm offset 0 1000
pwm oneshot 0 10
pwm enable 0
```

4.7 Counter

输入计数模式，支持计算输入波形的个数。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_counter_enable(pwm_device, channel);
result = rt_pwm_counter_get_result(pwm_device, channel, &cnt_res);
result = rt_pwm_counter_disable(pwm_device, channel);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。
- cnt_res：波形计数结果。

命令行测试指令：

```
pwm probe pwm0
pwm cnt_enable 0
pwm cnt_get_res 0
pwm cnt_disable 0
```

4.8 Frequency meter

频率计模式，支持计算输入波形的频率。

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set_freqency_meter(pwm_device, channel, delay_ms, &freq);
```

- name：设备名，如”pwm0“、”pwm1“等。
- channel：通道 ID。
- delay_ms：频率测量延迟时间，单位 ms。
- freq：频率计算结果，单位 Hz。

命令行测试指令：

```
pwm probe pwm0
pwm set_freq 0 1000
```

4.9 IR input

详见驱动 bsp/rockchip/common/drivers/drv_pwm_remotectl.c 及相关文档。